NumPy & Pandas Practice Guide

Data Analyst Roadmap: Days 9-15

Day	Module	Topics	Questions
Day 9	NumPy	Arrays, Indexing, Slicing	15
Day 10	NumPy	Math & Statistics Functions	15
Day 11	Pandas	Series, DataFrame Basics	15
Day 12	Pandas	Data Cleaning, Missing Values	15
Day 13	Pandas	GroupBy, Sort, Filter	20
Day 14	Pandas	Merge, Join, Concatenate	15
Day 15	Revision	Mini Test + EDA on Titanic Dataset	20
TOTAL			**115+ Questions**

How to Use This Guide

- Practice one day at a time, following on from your Python fundamentals (Days 1-8).
- Solve without looking at solutions first. Debug and understand errors as they come up.
- Use a real dataset (CSV file) wherever possible — Titanic, Iris, or your own sample data work well.
- Recommended time: 30-40 minutes daily for each day's questions.
- Install requirements first: pip install numpy pandas

DAY 9: NumPy - Arrays, Indexing, Slicing (15 Questions)

Refresh Topic: Arrays, Indexing, Slicing | Practice: 15 exercises | Task: Matrix Operations

Q1: Import NumPy and create a 1D array from a Python list of 10 integers. Print its shape, size, and dtype.

Q2: Create a NumPy array of zeros of shape (3,4), an array of ones of shape (2,5), and an identity matrix of size 4.

Q3: Create an array using np.arange(0, 50, 5) and reshape it into a suitable 2D shape.

Q4: Generate 10 evenly spaced numbers between 0 and 100 using np.linspace().

Q5: Create a 1D array of 20 numbers and extract elements from index 5 to 15 using slicing.

Q6: Create a 2D array (4x4) and access a specific row, a specific column, and a sub-matrix using slicing.

Q7: Create a 5x5 array of random integers between 1 and 100. Print the first two rows and last two columns.

Q8: Reverse a 1D NumPy array without using a loop.

Q9: Create a 3x3 array and use boolean indexing to extract all elements greater than 5.

Q10: Create two arrays of the same shape and stack them vertically and horizontally.

Q11: Split a 1D array of 12 elements into 3 equal parts using np.split().

Q12: Create a 1D array and use fancy indexing to select elements at positions [0, 2, 4, 6].

Q13: Create a 4x4 array and replace all values greater than 10 with 0 using conditional indexing.

Q14: Flatten a 2D array into 1D using both ravel() and flatten(), and explain the difference.

Q15: Create a NumPy array with np.arange(1,17) and reshape it into a 4x4 matrix. Transpose it.

DAY 10: NumPy - Math & Statistics Functions (15 Questions)

Refresh Topic: Math, Statistics | Practice: 15 exercises | Task: Array Analysis

Q1: Create two arrays of the same size and perform element-wise addition, subtraction, multiplication, and division.

Q2: Perform matrix multiplication (dot product) between two 2D arrays using np.dot() and the @ operator.

Q3: Create an array of 20 random integers and calculate its mean, median, standard deviation, and variance.

Q4: Find the minimum, maximum, and their index positions in a 1D array using argmin() and argmax().

Q5: Create a 2D array and calculate the sum along rows (axis=1) and along columns (axis=0).

Q6: Generate a random array of 15 values and sort it in ascending and descending order.

Q7: Create an array of angles in degrees and calculate their sine, cosine, and tangent using NumPy.

Q8: Compute the cumulative sum and cumulative product of an array using cumsum() and cumprod().

Q9: Create two arrays and calculate the correlation coefficient between them using np.corrcoef().

Q10: Generate a random array of 10 values and normalize it to a 0-1 range (min-max normalization).

Q11: Create a 1D array and count how many elements are above the mean value.

Q12: Use np.where() to replace negative numbers in an array with 0.

Q13: Create a random 5x5 matrix and calculate its determinant and inverse using np.linalg.

Q14: Generate two random arrays and calculate the Euclidean distance between them.

Q15: Create an array with some NaN values and calculate the mean while ignoring NaNs using np.nanmean().

DAY 11: Pandas - Series & DataFrame Basics (15 Questions)

Refresh Topic: Series, DataFrame | Practice: 15 exercises | Task: Load CSV

Q1: Import pandas and create a Series from a Python list with custom index labels.

Q2: Create a DataFrame from a dictionary containing columns: Name, Age, City, Salary (5 rows).

Q3: Load a CSV file into a DataFrame using pd.read_csv() and display the first 5 and last 5 rows.

Q4: Display the shape, columns, dtypes, and summary statistics (describe()) of a DataFrame.

Q5: Select a single column and multiple columns from a DataFrame.

Q6: Select rows using loc[] (label-based) and iloc[] (position-based) indexing.

Q7: Add a new calculated column to a DataFrame (e.g., Bonus = 10% of Salary).

Q8: Rename one or more column names in a DataFrame using rename().

Q9: Sort a DataFrame by a single column and by multiple columns.

Q10: Use info() to check data types and non-null counts of every column.

Q11: Filter rows in a DataFrame where a column value meets a condition (e.g., Age > 25).

Q12: Filter rows using multiple conditions combined with AND (&) and OR (|).

Q13: Drop a column and drop a row from a DataFrame using drop().

Q14: Reset the index of a DataFrame after filtering, using reset_index().

Q15: Convert a DataFrame column to a NumPy array and to a Python list.

DAY 12: Pandas - Data Cleaning & Missing Values (15 Questions)

Refresh Topic: Cleaning, Missing Values | Practice: 15 exercises | Task: Titanic Cleaning

Q1: Load the Titanic dataset (or any CSV with missing values) and count missing values in each column using isnull().sum().

Q2: Drop rows that contain any missing values using dropna().

Q3: Fill missing numeric values with the column mean using fillna().

Q4: Fill missing categorical values with the most frequent value (mode).

Q5: Detect and remove duplicate rows from a DataFrame using duplicated() and drop_duplicates().

Q6: Convert a column's data type (e.g., string to numeric) using astype() or pd.to_numeric().

Q7: Identify and handle outliers in a numeric column using the IQR (Interquartile Range) method.

Q8: Standardize inconsistent text data in a column (e.g., 'male', 'Male', 'MALE' -> 'Male').

Q9: Strip whitespace from string columns and convert them to lowercase.

Q10: Replace specific values in a column using replace() (e.g., replace 'NaN' strings with actual NaN).

Q11: Create a new column that flags rows with missing values (1 if any missing, else 0).

Q12: Use apply() with a custom function to clean or transform a column.

Q13: Bin a continuous numeric column into categories using pd.cut() (e.g., Age into child/adult/senior).

Q14: Check and correct the data type of a date column using pd.to_datetime().

Q15: Combine all the above techniques to fully clean a raw dataset and export the cleaned version to a new CSV.

DAY 13: Pandas - GroupBy, Sort, Filter (20 Questions)

Refresh Topic: GroupBy, Sort, Filter | Practice: 20 exercises | Task: Sales Summary

Q1: Group a sales DataFrame by 'Region' and calculate total sales per region using groupby().sum().

Q2: Group data by a categorical column and calculate mean, median, and count simultaneously using agg().

Q3: Group by two columns (e.g., Region and Product) and calculate total sales for each combination.

Q4: Find the top 5 highest-selling products using groupby() and sort_values().

Q5: Use value_counts() to find the frequency of each category in a column.

Q6: Sort a DataFrame by multiple columns with different ascending/descending order for each.

Q7: Filter a DataFrame to show only rows where a group's total exceeds a threshold.

Q8: Use groupby() with agg() to apply different aggregation functions to different columns.

Q9: Calculate the percentage contribution of each group to the overall total.

Q10: Use pivot_table() to create a summary table of sales by Region and Month.

Q11: Rank rows within each group using groupby() and rank().

Q12: Find the group with the maximum and minimum average value using groupby().

Q13: Filter groups that have more than N rows using groupby().filter().

Q14: Apply a custom aggregation function to grouped data using groupby().apply().

Q15: Create a cumulative sum within each group using groupby().cumsum().

Q16: Use nlargest() and nsmallest() to find the top and bottom N rows by a column.

Q17: Filter a DataFrame using the query() method instead of boolean indexing.

Q18: Group data by date (e.g., by Month) after converting a column to datetime, and calculate monthly totals.

Q19: Combine groupby() with multiple aggregations and rename the resulting columns.

Q20: Build a complete sales summary report: total, average, and count of orders by region and product category.

DAY 14: Pandas - Merge, Join & Concatenate (15 Questions)

Refresh Topic: Merge, Join | Practice: 15 exercises | Task: Combine Data

Q1: Create two small DataFrames (Employees and Departments) and merge them using pd.merge() on a common key.

Q2: Perform an inner join, left join, right join, and outer join between two DataFrames, and explain the difference in results.

Q3: Concatenate two DataFrames with the same columns vertically using pd.concat().

Q4: Concatenate two DataFrames side by side (horizontally) using pd.concat(axis=1).

Q5: Use join() to combine two DataFrames based on their index.

Q6: Merge two DataFrames on multiple key columns (e.g., Region and Year).

Q7: Handle overlapping column names during a merge using the suffixes parameter.

Q8: Merge a DataFrame with itself (self-join) to compare related rows (e.g., employee-manager pairs).

Q9: Identify rows that exist in one DataFrame but not the other using an outer join with an indicator column.

Q10: Combine three or more DataFrames from different sources into a single master DataFrame.

Q11: Merge a small lookup table (e.g., product ID to product name) into a larger sales DataFrame.

Q12: Use merge_ordered() or merge_asof() to combine time-series data from two sources.

Q13: After merging, handle any resulting missing values appropriately.

Q14: Verify a merge is correct by checking the row count before and after, and explain any discrepancies.

Q15: Combine everything: merge Sales, Products, and Customers tables into one clean report and export it to a new CSV.

DAY 15: REVISION - Mixed NumPy + Pandas Concepts (20 Questions)

Refresh Topic: Python + Pandas | Practice: Mini Test | Task: EDA on Titanic

Q1: Create a NumPy array of 100 random numbers and calculate all key statistics (mean, median, std, min, max) in one script.

Q2: Load the Titanic dataset and print a full data summary: shape, dtypes, missing values, and describe().

Q3: Clean the Titanic dataset: handle missing Age and Embarked values, drop irrelevant columns.

Q4: Find the survival rate (%) by gender using groupby().

Q5: Find the survival rate (%) by passenger class (Pclass) using groupby().

Q6: Create a new column 'AgeGroup' by binning Age into Child, Teen, Adult, Senior categories.

Q7: Find the average fare paid by passengers in each class.

Q8: Find the number of survivors and non-survivors combined by class and gender (two-level groupby).

Q9: Identify and handle outliers in the Fare column using the IQR method.

Q10: Create a pivot table showing average survival rate by Pclass and Sex.

Q11: Sort passengers by Fare in descending order and display the top 10.

Q12: Use NumPy to convert the Fare column into a normalized array (0 to 1 scale).

Q13: Merge the Titanic DataFrame with a small custom DataFrame mapping Pclass to a class label (e.g., 1='Upper').

Q14: Count how many passengers embarked from each port using value_counts().

Q15: Write 5 business insights based on your Titanic analysis (e.g., which group had the highest survival rate).

Q16: Export your cleaned and analyzed Titanic DataFrame to a new CSV file.

Q17: Build a small function that takes a DataFrame and column name, and returns a cleaning report (missing %, dtype, unique values).

Q18: Create a correlation matrix of all numeric columns in the Titanic dataset using NumPy/Pandas.

Q19: Compare two groups (e.g., male vs female fares) using mean, median, and standard deviation.

Q20: Summarize your Day 9-15 learning in a short README and upload your NumPy/Pandas practice notebook to GitHub.

Practice Tips & Guidelines

- Use Real Datasets: Practice on Titanic, Iris, or a sample sales CSV wherever possible.
- No Copy-Paste: Type the code yourself instead of copying from tutorials.
- Read the Docs: Get comfortable checking official NumPy and Pandas documentation.
- Print Often: Print intermediate outputs (shape, head(), dtypes) to understand what each step does.
- Understand Axis: Practice axis=0 (rows) vs axis=1 (columns) until it becomes intuitive.
- Test Edge Cases: Try empty DataFrames, all-NaN columns, and single-row data.
- Track Progress: Mark each question complete and note the time taken.
- Upload to GitHub: Save your Day 9-15 notebooks in your existing Python practice repository.
- Time Management: Days 9-14 (30-40 min each), Day 15 (45-60 min for full revision).
- Document Insights: For the Titanic EDA, always write down what the numbers mean in plain English.

Progress Summary

Day	Questions	Topics	Difficulty
9	15	NumPy Arrays & Indexing	Basic -> Intermediate
10	15	NumPy Math & Statistics	Intermediate
11	15	Pandas Series & DataFrame	Basic -> Intermediate
12	15	Pandas Cleaning & Missing Values	Intermediate
13	20	Pandas GroupBy, Sort, Filter	Intermediate -> Advanced
14	15	Pandas Merge & Join	Intermediate -> Advanced
15	20	Full Revision + Titanic EDA	Advanced

Next Steps: After completing Days 9-15 (NumPy & Pandas), you'll be ready to move into SQL (Days 16-30) as outlined in your Data Analyst Internship Playbook.